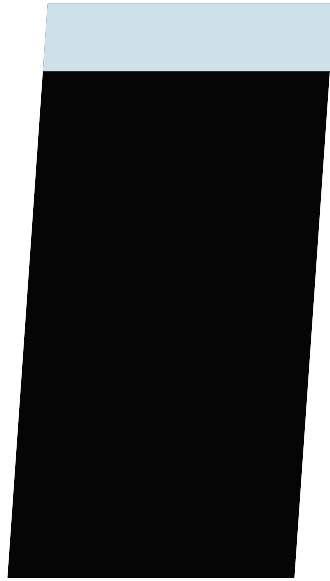




# call me maybe

## Introducción a la llamada de funciones en LLMs

Resumen: ¿Hablan los LLM el idioma de los ordenadores? Vamos a averiguarlo.  
Realizado en colaboración con @ldevelle, @pcamaren, @crfernan

Versión: 1.5

---

# Índice

|             |                                         |           |
|-------------|-----------------------------------------|-----------|
| <b>I</b>    | <b>Prólogo</b>                          | <b>2</b>  |
| <b>II</b>   | <b>Instrucciones sobre la IA</b>        | <b>3</b>  |
| <b>III</b>  | <b>Introducción</b>                     | <b>5</b>  |
| III.1       | ¿Qué es la llamada de funciones?        | 5         |
| III.2       | ¿Por qué es importante?                 | 6         |
| III.3       | El reto                                 | 6         |
| <b>IV</b>   | <b>Instrucciones comunes</b>            | <b>7</b>  |
| IV.1        | Reglas generales                        | 7         |
| IV.2        | Makefile                                | 7         |
| IV.3        | Directrices adicionales                 | 8         |
| IV.3.1      | Requisitos adicionales                  | 8         |
| IV.3.2      | Uso                                     | 9         |
| <b>V</b>    | <b>Parte obligatoria</b>                | <b>10</b> |
| V.1         | Resumen                                 | 10        |
| V.2         | Ficheros de entrada                     | 10        |
| V.3         | Interacción con el LLM                  | 12        |
| V.3.1       | El SDK del LLM                          | 12        |
| V.3.2       | El pipeline de generación               | 12        |
| V.3.3       | Entender la decodificación restringida  | 13        |
| V.4         | Formato del fichero de salida           | 14        |
| V.4.1       | Ejemplo de salida                       | 14        |
| V.4.2       | Reglas de validación                    | 14        |
| V.5         | Rendimiento y fiabilidad                | 15        |
| V.6         | Probar tu implementación                | 15        |
| <b>VI</b>   | <b>Requisitos del README</b>            | <b>16</b> |
| <b>VII</b>  | <b>Parte de bonus</b>                   | <b>18</b> |
| <b>VIII</b> | <b>Entrega y evaluación entre pares</b> | <b>19</b> |

## Capítulo I

# Prólogo

Los ingenieros romanos tallaban en tablillas de piedra cuadrículas perfectas para llevar la cuenta de los cargamentos de grano por todo el imperio, de modo que ningún envío se perdiera por culpa de una mala caligrafía. En el siglo XIX, los marineros registraban las corrientes oceánicas en tablas estructuradas tan precisas que algunas todavía se usan hoy en la navegación.

Las primeras previsiones meteorológicas se enviaban como telegramas en un código fijo: unos pocos números capaces de describir todo un cielo. En los años sesenta, el control de misiones de la NASA trabajaba con diagramas de flujo plastificados que les indicaban exactamente qué significaba cada luz parpadeante, sin importar quién estuviera de turno. Los escáneres de códigos de barras de los supermercados traducían rayas en blanco y negro a datos de inventario mucho antes de que la mayoría de los hogares tuviera un ordenador. Incluso los apicultores han utilizado durante décadas formularios estandarizados, anotando la salud de la colmena, el flujo de néctar y el linaje de la reina en casillas diminutas que solo ellos podían leer de un vistazo.

Los seres humanos siempre han construido estructuras para hacer que la información sea fiable, compartible y utilizable. Lo cual nos lleva hasta hoy, donde el objetivo es hacer que la IA hable el idioma de los ordenadores.

## Capítulo II

# Instrucciones sobre la IA

### ● Contexto

Durante tu proceso de aprendizaje, la IA puede ayudarte en muchas tareas distintas. Dedicar tiempo a explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, acércate siempre a ellas con cautela y evalúa los resultados de forma crítica. Ya se trate de código, documentación, ideas o explicaciones técnicas, nunca podrás estar del todo seguro de que tu pregunta estaba bien formulada o de que el contenido generado es exacto. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

### ● Mensaje principal

- Usa la IA para reducir las tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting —tanto de programación como ajenas a ella— que beneficiarán tu carrera profesional futura.
- Aprende cómo funcionan los sistemas de IA para anticipar y evitar mejor los riesgos, sesgos y problemas éticos habituales.
- Sigue desarrollando tanto habilidades técnicas como habilidades transversales trabajando con tus compañeros.
- Utiliza únicamente contenido generado por IA que entiendas plenamente y del que puedas responsabilizarte.

### ● Reglas para el estudiante:

- Deberías dedicar tiempo a explorar las herramientas de IA y entender cómo funcionan, para poder usarlas de forma ética y reducir posibles sesgos.
- Deberías reflexionar sobre tu problema antes de hacer prompting: esto te ayuda a escribir prompts más claros, detallados y relevantes, con un vocabulario preciso.
- Deberías desarrollar el hábito de comprobar, revisar, cuestionar y probar de forma sistemática todo lo generado por IA.
- Deberías buscar siempre la revisión de tus compañeros: no confíes únicamente en tu propia validación.

### ● Resultados de la fase:

- Desarrollar habilidades de prompting tanto de propósito general como específicas de un dominio.

- 
- Aumentar tu productividad con un uso eficaz de las herramientas de IA.
  - Seguir reforzando el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

## ● Comentarios y ejemplos:

- Te encontrarás con frecuencia en situaciones —exámenes, evaluaciones y más— en las que tendrás que demostrar una comprensión real. Prepárate y sigue desarrollando tanto tus habilidades técnicas como las interpersonales.
- Explicar tu razonamiento y debatir con tus compañeros a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a dar respuestas genéricas. Tus compañeros, que comparten tu entorno, pueden ofrecerte ideas más relevantes y precisas.
- Mientras que la IA tiende a generar la respuesta más probable, tus compañeros pueden aportar perspectivas alternativas y matices valiosos. Apóyate en ellos como punto de control de calidad.

### ✓ Buena práctica:

Le pregunto a la IA: «¿Cómo pruebo una función de ordenación?». Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

### ✗ Mala práctica:

Le pido a la IA que escriba una función entera, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no sé explicar qué hace ni por qué. Pierdo credibilidad y suspendo mi proyecto.

### ✓ Buena práctica:

Uso la IA para ayudarme a diseñar un analizador sintáctico. Después reviso la lógica paso a paso con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y comprendido por completo.

### ✗ Mala práctica:

Dejo que Copilot genere el código de una parte clave de mi proyecto. Compila, pero no sé explicar cómo gestiona las pipes. Durante la evaluación, no consigo justificarlo y suspendo mi proyecto.

## Capítulo III

# Introducción

### III.1 ¿Qué es la llamada de funciones?

Los grandes modelos de lenguaje (LLM) son potentes a la hora de entender y generar lenguaje humano, pero no producen de forma natural una salida estructurada y ejecutable por una máquina. La llamada de funciones (function calling) salva esa distancia traduciendo peticiones en lenguaje natural a llamadas de función precisas con argumentos tipados.

Considera este ejemplo: de lenguaje natural a llamada de función.

```
User: "What is the sum of 40 and 2?"

Traditional LLM: "The sum of 40 and 2 is 42."

Function Calling System:
{
  "function": "add_numbers",
  "arguments": {"a": 40, "b": 2}
}
```

El sistema de llamada de funciones no responde la pregunta directamente. En su lugar, proporciona las **herramientas** para resolverla: el nombre de función correcto y los argumentos adecuados con los tipos apropiados.

### III.2 ¿Por qué es importante?

La llamada de funciones permite a los LLM:

- **Interactuar con sistemas externos:** llamar a APIs, consultar bases de datos, controlar dispositivos.
- **Ejecutar código:** realizar cálculos, transformaciones de datos, operaciones sobre ficheros.
- **Encadenar operaciones:** descomponer tareas complejas en pasos ejecutables.
- **Proporcionar salida estructurada:** generar JSON, XML u otros formatos legibles por máquina.
- **Extraer datos estructurados de texto no estructurado:** por ejemplo, dado un libro extenso, extraer campos como {nombre del protagonista, sexo del protagonista, edad del protagonista}.

---

Esta tecnología impulsa los asistentes de IA modernos, las herramientas de generación de código y los agentes autónomos, además de habilitar tareas como la extracción automática de información y la estructuración de conocimiento a partir de texto en bruto.

### III.3 El reto

Los modelos de lenguaje pequeños (como el modelo de 0,6 B de parámetros que vas a usar) son tristemente conocidos por su poca fiabilidad al generar salida estructurada. Cuando se les pide que produzcan JSON, pueden acertar solo el 30 % de las veces. Y sin embargo, los sistemas en producción alcanzan una fiabilidad del 99 % o más con esos mismos modelos pequeños.

**¿Cómo?** La respuesta está en la **decodificación restringida** (constrained decoding): una técnica que guía la salida del modelo token a token para garantizar una estructura válida, sin depender únicamente del prompting.

---

## Capítulo IV

# Instrucciones comunes

### IV.1 Reglas generales

- Tu proyecto debe estar escrito en **Python 3.10 o posterior**.
- Tu proyecto debe cumplir el estándar de estilo **flake8**.
- Tus funciones deben gestionar las excepciones con elegancia para evitar caídas. Usa bloques `try-except` para gestionar posibles errores. Prefiere los gestores de contexto para recursos como ficheros o conexiones, con el fin de garantizar una limpieza automática. Si tu programa se cae por excepciones no gestionadas durante la revisión, se considerará no funcional.
- Todos los recursos (por ejemplo, descriptores de fichero, conexiones de red) deben gestionarse correctamente para evitar fugas. Usa gestores de contexto siempre que sea posible para un manejo automático.
- Tu código debe incluir anotaciones de tipo (type hints) para los parámetros de función, los tipos de retorno y las variables cuando proceda (usando el módulo `typing`). Usa `mypy` para la comprobación estática de tipos. Todas las funciones deben pasar `mypy` sin errores.
- Incluye docstrings en funciones y clases siguiendo PEP 257 (por ejemplo, estilo Google o NumPy) para documentar el propósito, los parámetros y los valores de retorno.

### IV.2 Makefile

Incluye un `Makefile` en tu proyecto para automatizar las tareas habituales. Debe contener las siguientes reglas (el `lint` obligatorio implica los flags indicados; se recomienda encarecidamente probar `-strict` para una comprobación más exhaustiva):

- **install**: instalar las dependencias del proyecto usando `pip`, `uv`, `pipx` o cualquier otro gestor de paquetes de tu elección.
- **run**: ejecutar el script principal de tu proyecto (por ejemplo, mediante el intérprete de Python).
- **debug**: ejecutar el script principal en modo depuración usando el depurador integrado de Python (por ejemplo, `pdb`).
- **clean**: eliminar ficheros temporales o cachés (por ejemplo, `__pycache__`, `.mypy_cache`) para mantener limpio el entorno del proyecto.



- 
- **lint**: ejecutar los comandos `flake8 .` y `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
  - **lint-strict** (opcional): ejecutar los comandos `flake8 .` y `mypy . --strict`

## IV.3 Directrices adicionales

- Crea programas de prueba para verificar la funcionalidad del proyecto (no se entregan ni se califican). Usa frameworks como `pytest` o `unittest` para las pruebas unitarias, cubriendo los casos límite.
- Incluye un fichero `.gitignore` para excluir los artefactos de Python.
- Se recomienda usar entornos virtuales (por ejemplo, `venv` o `conda`) para aislar las dependencias durante el desarrollo.

Si se aplican requisitos adicionales específicos del proyecto, se indicarán inmediatamente debajo de esta sección.

### IV.3.1 Requisitos adicionales

- Todas las clases deben usar **pydantic** para la validación.
- Puedes usar los paquetes **numpy** y **json**.
- El uso de **dspy** (o cualquier paquete similar) está totalmente prohibido, incluidos `pytorch`, el paquete de `huggingface`, `transformers`, `outlines`, etc.
- Debes usar los siguientes modelos:
  - **Qwen/Qwen3-0.6B** (por defecto)
  - Puedes usar otros modelos siempre que tu proyecto funcione con **Qwen/Qwen3-0.6B**.
- La función que se va a llamar debe elegirse usando el LLM, no con heurísticas ni con ningún otro tipo de magia medieval.
- Está prohibido usar cualquier método o atributo privado del paquete `llm_sdk`.
- Debes crear un entorno virtual e instalar los paquetes **numpy** y **pydantic** usando `uv`. Para usar `llm_sdk`, puedes copiarlo en el mismo directorio en el que está `src`.
- Tanto la persona que revise como la moulinette se limitarán a ejecutar `uv sync`.
- Todos los errores deben gestionarse con elegancia. Tu programa nunca debe caer de forma inesperada y siempre debe proporcionar mensajes de error claros al usuario.

### IV.3.2 Uso

Tu programa debe ejecutarse mediante el siguiente comando (donde `src` es la carpeta que contiene tus ficheros):

Ejecución del programa

```
uv run python -m src [--functions_definition <function_definition_file>] [--input  
<input_file>] [--output <output_file>]
```

① Por defecto, el programa leerá los ficheros de entrada del directorio `data/input/` y escribirá la salida en el directorio `data/output/`. Opcionalmente, puedes especificar rutas personalizadas usando los argumentos `--input` y `--output`. Por ejemplo:

```
uv run python -m src  
--functions_definition data/input/functions_definition.json  
--input data/input/function_calling_tests.json  
--output data/output/function_calls.json
```

## Capítulo V

# Parte obligatoria

### V.1 Resumen

En este proyecto crearás una herramienta de llamada de funciones que traduce prompts en lenguaje natural a llamadas de función estructuradas. Dada una pregunta como «¿Cuál es la suma de 40 y 2?», tu solución no debe devolver 42, sino proporcionar:

- El nombre de la función: `fn_add_numbers`
- Los argumentos: `{"a": 40, "b": 2}`

Tu implementación debe usar **decodificación restringida** para garantizar una salida JSON 100 % válida, asegurando una fiabilidad casi perfecta incluso con un modelo pequeño de 0,6 B de parámetros.

### V.2 Ficheros de entrada

Tu solución procesará dos ficheros de entrada situados en el directorio `data/input/`:

- `function_calling_tests.json`: contiene un array JSON de prompts en lenguaje natural que tu sistema debe procesar.

Ejemplo: `function_calling_tests.json`

```
[
  {
    "prompt": "What is the sum of 2 and 3?"
  },
  {
    "prompt": "What is the sum of 265 and 345?"
  },
  {
    "prompt": "Greet shrek"
  },
  {
    "prompt": "Greet john"
  },
  {
    "prompt": "Reverse the string 'hello'"
  },
]
```

```
...  
]
```

- `functions_definition.json`: contiene las funciones disponibles que tu sistema puede llamar. Cada función incluye:
  - Nombre de la función
  - Nombres y tipos de los argumentos
  - Tipo de retorno
  - Descripción

Ejemplo: `functions_definition.json`

```
[  
  {  
    "name": "fn_add_numbers",  
    "description": "Add two numbers together and return their sum.",  
    "parameters": {  
      "a": {  
        "type": "number"  
      },  
      "b": {  
        "type": "number"  
      }  
    },  
    "returns": {  
      "type": "number"  
    }  
  },  
  {  
    "name": "fn_greet",  
    "description": "Generate a greeting message for a person by name.",  
    "parameters": {  
      "name": {  
        "type": "string"  
      }  
    },  
    "returns": {  
      "type": "string"  
    }  
  },  
  {  
    "name": "fn_reverse_string",  
    "description": "Reverse a string and return the reversed result.",  
    "parameters": {  
      "s": {  
        "type": "string"  
      }  
    },  
    "returns": {  
      "type": "string"  
    }  
  }  
],
```

```
...
]
```

⚠ Estos ejemplos establecen el nivel de complejidad esperado. Sin embargo, tu solución se probará con prompts y conjuntos de funciones distintos. Debes implementar una gestión de errores de JSON adecuada para los ficheros de entrada, ya que pueden contener JSON no válido o faltar por completo.

## V.3 Interacción con el LLM

### V.3.1 El SDK del LLM

Junto a este proyecto encontrarás una clase envoltorio (wrapper) **Small\_LLM\_Model** en el paquete `llm_sdk` que puedes usar para interactuar con el LLM.

El SDK proporciona varios métodos esenciales:

- `get_logits_from_input_ids(input_ids: List[int]) -> List[float]`  
Toma una lista de IDs de token y devuelve los logits producidos por el modelo LLM.
- `get_path_to_vocab_file() -> str`  
Devuelve la ruta al fichero de vocabulario que contiene la correspondencia entre IDs de token y tokens.
- `encode(text: str) -> Tensor`  
Codifica una cadena de texto en un tensor de IDs de token usando el tokenizer del modelo.
- `decode(token_ids: List[int]) -> str (opcional)`  
Decodifica opcionalmente una lista de IDs de token de vuelta a una cadena de texto.

### V.3.2 El pipeline de generación

El proceso de generación del LLM sigue estos pasos:

1. **Prompt:** tu pregunta en lenguaje natural.  
Ejemplo: «What is the sum of 2 and 3?»
2. **Tokenización:** el texto se divide en unidades de subpalabra (tokens). A diferencia de una simple división por palabras, los tokenizers suelen incluir los espacios iniciales, manejar la puntuación y dividir las palabras en componentes más pequeños usando algoritmos como BPE o SentencePiece.  
Ejemplo (realista): ["What", "Ġis", "Ġthe", "Ġsum", "Ġof", "Ġ2", "Ġand", "Ġ3", "?"]  
Nota: el símbolo «Ġ» indica un espacio precedente; los tokenizers reales conservan estos detalles para reconstruir el texto con exactitud.
3. **Input IDs:** los tokens se convierten en IDs numéricos que el modelo entiende.  
Ejemplo (ilustrativo): [892, 318, 262, 4771, 286, 16, 290, 17, 30]

4. **Procesamiento del LLM:** el modelo procesa estos números a través de su red neuronal.
5. **Logits:** el modelo produce puntuaciones de probabilidad para cada posible token siguiente.  
Ejemplo: token\_5: 0.001, token\_42: 0.85, token\_100: 0.02, ...
6. **Selección de token:** el siguiente token se elige a partir de estas probabilidades, normalmente el de mayor puntuación.  
En esta etapa se pueden aplicar técnicas como la **decodificación restringida** para limitar las opciones de token y asegurar que las salidas sigan una estructura concreta, como generar JSON 100 % válido.

**Importante:** este proceso se repite token a token. Cada token generado se añade al prompt, y los pasos 2 a 6 se repiten hasta que se genera la respuesta completa.

#### Vista simplificada:

```
Prompt -> Tokenization -> Input IDs -> LLM -> Logits -> Next Token Selection
```

### V.3.3 Entender la decodificación restringida

Los modelos de lenguaje generan texto un token cada vez. En cada paso, el modelo produce una distribución de probabilidad (logits) sobre todos los posibles tokens siguientes. Normalmente, muestrearías de esta distribución o elegirías el token de mayor probabilidad.

La **decodificación restringida** interviene en este proceso modificando los logits antes de la selección de token:

1. El modelo produce logits para todos los posibles tokens.
2. Identificas qué tokens mantendrían tanto una estructura JSON válida como la conformidad con el esquema esperado.
3. Estableces los logits de los tokens no válidos (los que rompen el esquema o la estructura) en menos infinito.
4. Muestras únicamente entre los tokens válidos restantes.

En este proyecto, la decodificación restringida no solo debe garantizar un JSON sintácticamente válido, sino también hacer cumplir un esquema concreto. Por ejemplo, si un campo está restringido a un número en `functions_definition.json`, el decodificador limita la selección de tokens a valores que satisfagan un entero o un número de coma flotante, preservando tanto la validez del JSON como la conformidad con el esquema. Esto garantiza que cada token generado mantenga la validez tanto estructural como semántica, haciendo cumplir el esquema requerido. Como resultado, el JSON producido es 100 % recuperable y siempre puede analizarse sin errores.

⚠ Tu solución NO debe basarse en que el modelo produzca espontáneamente un JSON correcto a partir de un prompt. Pasar al modelo las definiciones de funciones y confiar en obtener una salida estructurada no es fiable, y no es la habilidad que esperamos que desarrolles aquí.

① Piensa en cómo puedes usar el fichero JSON de vocabulario para mapear entre tokens y sus representaciones como cadena. Esto es crucial para determinar qué tokens son válidos en cada paso de generación.

## V.4 Formato del fichero de salida

Tu programa producirá un único fichero JSON: `data/output/function_calling_results.json`. Por cada prompt, añade un objeto JSON a este fichero. Cada objeto del array debe contener exactamente las siguientes claves:

- `prompt` (string): la petición original en lenguaje natural.
- `name` (string): el nombre de la función que se va a llamar.
- `parameters` (object): todos los argumentos requeridos con los tipos correctos.

### V.4.1 Ejemplo de salida

```
[
  {
    "prompt": "What is the sum of 2 and 3?",
    "name": "fn_add_numbers",
    "parameters": {"a": 2.0, "b": 3.0}
  },
  {
    "prompt": "Reverse the string 'hello'",
    "name": "fn_reverse_string",
    "parameters": {"s": "hello"}
  }
]
```

### V.4.2 Reglas de validación

- El fichero debe ser JSON válido (sin comas finales, sin comentarios).
- Las claves y los tipos deben coincidir exactamente con el esquema de `functions_definition.json`.
- No se permiten claves adicionales ni texto en prosa en ningún lugar de la salida.
- Todos los argumentos requeridos deben estar presentes.
- Los tipos de los argumentos deben coincidir con la definición de la función (number, string, boolean, etc.).

⚠ Los ficheros de entrada proporcionados pueden cambiar durante la evaluación entre pares. No codifiques soluciones a fuego (hardcode) basándote en los ejemplos proporcionados.

## V.5 Rendimiento y fiabilidad

Tu implementación debería alcanzar:

- **Precisión casi perfecta:** 90 % o más de acierto en la selección de función y la extracción de argumentos.
- **JSON 100 % válido:** toda salida debe poder analizarse y cumplir el esquema.
- **Velocidad razonable:** procesar todos los prompts de prueba en menos de 5 minutos en hardware estándar.
- **Gestión de errores robusta:** manejar con elegancia entradas mal formadas, ficheros ausentes y casos límite.

① El modelo Qwen3-0.6B tiene solo 500 millones de parámetros y, sin embargo, con una decodificación restringida adecuada, puede alcanzar una fiabilidad comparable a la de modelos mucho más grandes. Esto demuestra el poder de la guía estructural frente al tamaño puro del modelo.

## V.6 Probar tu implementación

Para verificar que tu solución funciona correctamente:

1. Asegúrate de que los ficheros de entrada están en el directorio `data/input/`.
2. Ejecuta: `uv run python -m src [-functions_definition <function_definition_file>] [-input <input_file>] [-output <output_file>]`
3. Comprueba que se crea `output/function_calling_results.json`.
4. Valida la estructura y el contenido del JSON.
5. Verifica que los nombres de función y los tipos de argumento coinciden con las definiciones.

⚠ Prueba con varios casos límite: cadenas vacías, números grandes, caracteres especiales, tipos incorrectos, prompts ambiguos y funciones con múltiples parámetros.



## Capítulo VI

# Requisitos del README

Se debe proporcionar un fichero `README.md` en la raíz de tu repositorio Git. Su propósito es permitir que cualquier persona no familiarizada con el proyecto (compañeros, personal, reclutadores, etc.) entienda rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.

El `README.md` debe incluir al menos:

- La primera línea debe ir en cursiva y decir: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]]*.
- Una sección «**Description**» que presente claramente el proyecto, incluidos su objetivo y una breve descripción general.
- Una sección «**Instructions**» que contenga toda la información relevante sobre compilación, instalación y/o ejecución.
- Una sección «**Resources**» que enumere las referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción de cómo se usó la IA, especificando para qué tareas y en qué partes del proyecto.
- **Pueden requerirse secciones adicionales según el proyecto** (por ejemplo, ejemplos de uso, lista de características, decisiones técnicas, etc.).

Cualquier añadido requerido se indicará explícitamente a continuación.

Para este proyecto, el `README.md` también debe incluir:

- **Explicación del algoritmo:** describe en detalle tu enfoque de decodificación restringida.
- **Decisiones de diseño:** explica las decisiones clave de tu implementación.
- **Análisis de rendimiento:** comenta la precisión, la velocidad y la fiabilidad de tu solución.
- **Retos afrontados:** documenta las dificultades encontradas y cómo las resolviste.
- **Estrategia de pruebas:** describe cómo validaste tu implementación.
- **Ejemplo de uso:** proporciona ejemplos claros de ejecución de tu programa.

❗ Tu README debe estar escrito en inglés.

## Capítulo VII

# Parte de bonus

Se valorarán características de bonus (opcionales, no necesarias para aprobar):

- Soporte para múltiples modelos LLM más allá de Qwen/Qwen3-0.6B.
- Reimplementar el tokenizer: evitar el uso directo de `encode` y `decode` en el código principal, usando en su lugar `get_logits_from_input_ids` y `get_path_to_vocab_file`.
- Mecanismos avanzados de recuperación ante errores.
- Optimizaciones de rendimiento (caché, batching).
- Conjunto de pruebas exhaustivo.
- Visualización del proceso de generación.
- Soporte para argumentos de función anidados y complejos.
- Implementación pública de los métodos `encode` y (opcionalmente) `decode` del tokenizer.
- Demostración de cómo la codificación y la decodificación se integran con la decodificación restringida.

Las características de bonus deben estar implementadas y funcionando, no solo descritas en el `README.md`. Se te podrá pedir que las demuestres durante la evaluación.

## Capítulo VIII

# Entrega y evaluación entre pares

Entrega tu tarea en tu repositorio Git como de costumbre. Solo se revisará el trabajo que esté dentro de tu repositorio durante la defensa. No dudes en volver a comprobar los nombres de tus ficheros para asegurarte de que son correctos.

Tu repositorio debe contener:

- El directorio `src/` con tu implementación.
- `pyproject.toml` y `uv.lock` para la gestión de dependencias.
- El directorio `llm_sdk/` (copiado del paquete proporcionado).
- El directorio `data/input/` con ficheros de prueba (para la demostración).
- `README.md` con documentación completa.
- Cualquier fichero adicional necesario para ejecutar tu solución.

⚠ No incluyas el directorio `output/` en tu repositorio. Se generará durante la evaluación entre pares.

Durante la evaluación, ocasionalmente se puede solicitar una breve **modificación del proyecto**. Esto podría implicar un pequeño cambio de comportamiento, unas pocas líneas de código que escribir o reescribir, o una característica fácil de añadir.

Aunque este paso puede **no ser aplicable a todos los proyectos**, debes estar preparado para él si se menciona en las directrices de evaluación.

Este paso pretende verificar tu comprensión real de una parte concreta del proyecto. La modificación puede realizarse en cualquier entorno de desarrollo que elijas (por ejemplo, tu configuración habitual), y debería ser factible en unos pocos minutos, salvo que se defina un plazo específico como parte de la evaluación.

Por ejemplo, se te puede pedir que hagas una pequeña actualización de una función o de un script, que modifiques una visualización o que ajustes una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las directrices de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.